

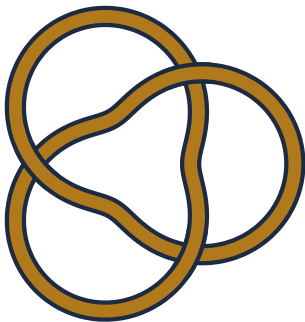


nibart

Calligraphy, knots and ornament — a MetaPost-like pen for Typst

Reference manual Version 0.3.0 · Typst ≥ 0.15 · MIT

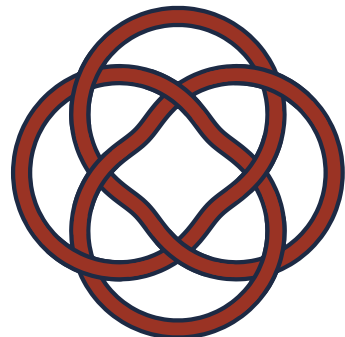
Author FERGOUS Abdelhak <@fergousA> · github.com/fergousA/nibart



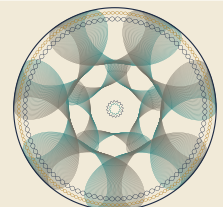
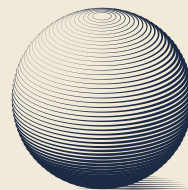
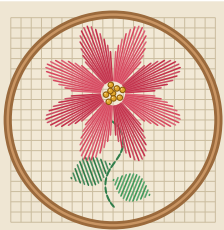
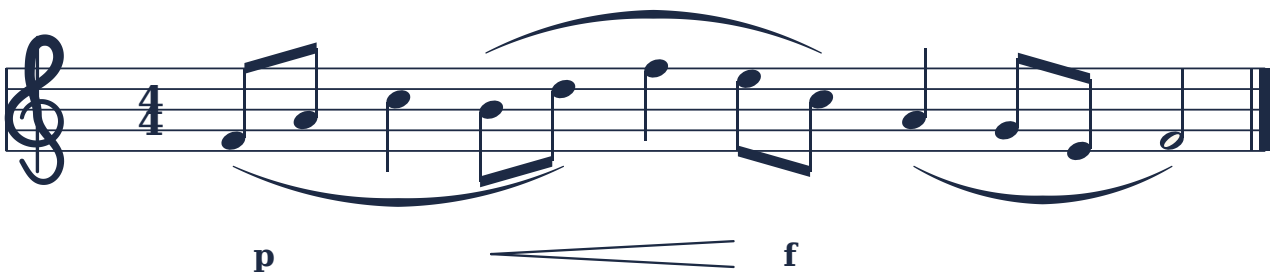
3 crossings



5 crossings



8 crossings



Contents

Getting started	3
What nibart does	3
Installation	3
Quick start	3
Concepts	4
How to read this manual	4
Paths and geometry	5
Building paths	5
Measuring and querying	12
Cutting and editing	18
Transformations	23
Pens	27
Fixed pens	27
The calligraphic pen and its options	29
Drawing	32
Strokes	32
Fills, dots, labels	34
Figures	36
Knots, ornaments and decorations	38
Knots and links	38
Calligraphic decorations	40
Version	41
Recipes	43
Handwriting-like loop with a broad nib	43
A curved arrow	43
A string of beads on a closed curve	43
Over/under crossings by hand	43
Migrating a drawing written for nibst	44
Notes and limits	45
Credits and licence	46
Index of functions	47

Getting started

What nibart does

`nibart` draws strokes the way Metafont and MetaPost do: you describe a **path** (Hobby's smooth-curve algorithm, directions, tensions, curls...) and a **pen** (circle, ellipse, broad-edged nib, polygon, a pen whose shape changes along the path), and `nibart` computes the exact outline swept by the pen. The result is an ordinary Typst vector shape. On top of this engine, the package offers **knots and interlaced work**, **calligraphic decorations** (braces, pointed-pen strokes, multi-line nibs) and **path surgery** (cutting, offsetting, placing things along a curve).

The geometry is computed by a small Rust plugin compiled to WebAssembly (`nibart.wasm`) shipped inside the package. No font, no external program, no network.

Installation

```
#import "@preview/nibart:0.3.0" as nib
```

Typst Universe: nothing to install. A local edition (zip with `install.sh` / `install.ps1`) installs the same package under the `@local` namespace.

Prefer the qualified form `nib.xxx`: the package exports generic names (`draw`, `cubic`, `reverse`, `pressure`, `shifted`...) that would shadow your own definitions with `import "...": *`. You can also import only what you need: `#import "@preview/nibart:0.3.0": mp-path, nib-stroke, broadnib`.

Quick start

One path, one pen, one call:

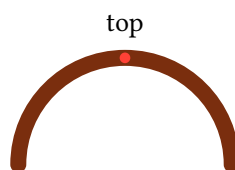
```
#import "@preview/nibart:0.3.0" as nib
```

```
#nib.nib-stroke(  
  nib.mp-path("(0,0)..(40,30)..(80,0)..(120,30)"),  
  pen: nib.broadnib(10pt, t: 2.5pt, angle: 30deg),  
)
```



For more control, build the drawables yourself and put them in a figure:

```
#nib.mp-fig(  
  nib.draw(nib.mp-path("(0,0){up}..(40,40)..(80,0)"),  
    pen: nib.pencircle(6pt), fill: rgb("#7a2e0e")),  
  nib.mp-dot((40pt, 40pt), pen: nib.pencircle(4pt), fill: red),  
  nib.mp-label([top], (40pt, 50pt), align: bottom,  
    pad: 4pt,  
)
```



Concepts

- **Axes.** As in MetaPost, **y points up** and angles are counter-clockwise. Inside an `mp-fig` the origin is where you set it (by default the content is fitted to its bounding box).
- **Units.** Anything you pass to a function can be a Typst length or angle (`3mm`; `1em` is refused because it depends on the context) or a plain number (read as pt, or as degrees for an angle). Path strings (`mp-path`) read plain numbers times `unit` (default `1pt`).
- **Paths** are opaque values (a list of Bézier segments). They are created by `mp-path` (MetaPost syntax), `mp-path-pts`, `cubic`, `straight`, `polyline`, `cubics`; combined by `path-join`; transformed by `shifted`, `scaled`, `rotated`...
- **Times.** A path of `n` segments has times from `0` to `n`. Time `2.5` is the middle of the third segment — as with MetaPost's `point 2.5 of p`.
- **Pens** are plain dictionaries: fixed (`penellipse`, `pencircle`, `broadnib`, `penpoly`, `pensquare`, `penrazor`) or variable (`nibpen`).
- **Drawables** — created by `draw`, `stroke-items`, `mp-fill`, `mp-dot`, `mp-label`, `mp-skeleton`, `mp-group`, and by the ornament functions of the second part — are handed to `mp-fig`, which returns an ordinary `box`. `nib-stroke` is the shortcut for one stroke.
- **Shapes, not strokes.** `draw` and friends return **filled** shapes (the region swept by the pen), so `fill` is the colour of the ink.

How to read this manual

The reference below describes every public function in the same way: its **signature** with all the parameters and their defaults; a **table of parameters** (name, type, default, meaning — “required” means no default); what it **returns**; and a detailed **example**, with the code on the left and the live result on the right. Comments in the examples explain each step.

The examples are written in **code mode** and use the names **without** the `nib.` prefix, as if after `#import "@preview/nibart:0.3.0": *`. In your document, either import the names you need or prefix them with `nib.`, and put the code in `#{ ... }` or in a function body. The value of the last expression is what appears on the right. An **index of all functions**, with page numbers, closes the manual.

Paths and geometry

Building paths

A **path** is an opaque value: a list of cubic Bézier segments, open or cyclic. Every function below returns one. Coordinates are Typst lengths (plain numbers mean pt); **y points up**.

mp-path

```
mp-path(spec, unit: 1pt)
```

Parses a path written in the MetaPost language and computes the smooth curve with Hobby's algorithm. Supported syntax: points `(x,y)`; joins `..` (Hobby curve), `...` (treated as `..`) and `--` (straight line); direction specifiers `{up}`, `{down}`, `{left}`, `{right}`, `{dir 45}`, `{x,y}`, `{curl 2}`; `tension a` and `tension a and b`; `controls P and controls P and Q`; `cycle` to close the path.

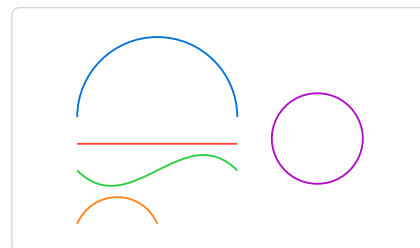
PARAMETERS

name	type	default	description
<code>spec</code>	<code>string</code>	required	The path specification.
<code>unit</code>	<code>length</code>	<code>1pt</code>	Unit that multiplies every plain number of <code>spec</code> .

RETURNS a path.

EXAMPLE

```
// four specifications, drawn as skeletons
let a = mp-path("(0,0)..(30,30)..
(60,0)") // plain Hobby curve
// imposed direction + tension
let b = mp-path("(0,-10){right}..tension 2..(60,-10)")
// explicit Bézier controls
let c = mp-path("(0,-20)..controls (20,-40) and (40,0)..
(60,-20)")
let d = mp-path("(75,0)..(105,0)..
(90,-25)..cycle") // closed path
// numbers are multiplied by `unit`
let e = mp-path("(0,0)..(3,2)..(6,0)", unit: 5pt)
mp-fig(
  mp-skeleton(a, stroke: 0.7pt + blue),
  mp-skeleton(b, stroke: 0.7pt + red),
  mp-skeleton(c, stroke: 0.7pt + green),
  mp-skeleton(d, stroke: 0.7pt + purple),
  mp-skeleton(shifted(e, 0pt, -40pt), stroke: 0.7pt +
orange),
  pad: 4pt)
```



mp-path-pts

```
mp-path-pts(pts, cycle: false, tension: 1, curl: none)
```

Smooth (Hobby) path through a list of points. Handy when the points are computed in a loop and you do not want to build a specification string.

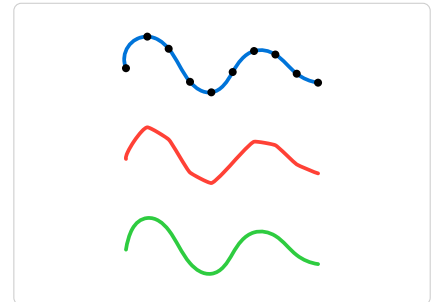
PARAMETERS

name	type	default	description
pts	array of points	required	Points (x, y), each coordinate a length.
cycle	bool	false	Close the path smoothly.
tension	number	1	Tension of every join (as in <code>..tension t..</code>); larger = tighter curve.
curl	number or none	none	Curl at the first point of an open path (<code>{curl c}</code>); 0 = flat ends.

RETURNS a path.

EXAMPLE

```
// ten points of a damped wave
let pts = range(10).map(i =>
  (i * 8pt, 14pt * calc.sin(i * 70deg) * calc.pow(0.9,
i)))
// same points, three settings, stacked
// les mêmes points, trois réglages, empilés
let loose = mp-path-pts(pts)
let tight = shifted(mp-path-pts(pts, tension: 3), 0pt,
-34pt)
let flat = shifted(mp-path-pts(pts, curl: 0), 0pt, -68pt)
mp-fig(
  draw(loose, pen: pencircle(1.4pt), fill: blue),
  draw(tight, pen: pencircle(1.4pt), fill: red),
  draw(flat, pen: pencircle(1.4pt), fill: green),
  ..pts.map(q => mp-dot(q, pen: pencircle(3pt))),
  pad: 4pt)
```



cubic

`cubic(a, b, c, d)`

One cubic Bézier segment from `a` to `d` with control points `b` and `c`.

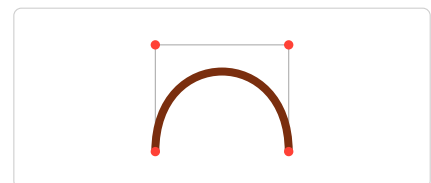
PARAMETERS

name	type	default	description
a	point	required	Start point (x, y).
b	point	required	First control point.
c	point	required	Second control point.
d	point	required	End point.

RETURNS an open path of one segment.

EXAMPLE

```
let (a, b, c, d) = ((0pt, 0pt), (0pt, 40pt), (50pt,
40pt), (50pt, 0pt))
let p = cubic(a, b, c, d)
mp-fig(
  // control polygon
  mp-skeleton(polyline((a, b, c, d)), stroke: 0.4pt +
gray),
  draw(p, pen: pencircle(3pt), fill: rgb("#7a2e0e")),
  ..(a, b, c, d).map(q => mp-dot(q, pen:
pencircle(3.5pt), fill: red)),
  pad: 5pt)
```



straight

```
straight(a, b)
```

One straight segment, stored as a degenerate cubic so that it combines freely with curves.

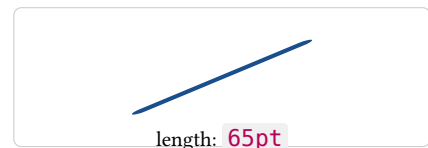
PARAMETERS

name	type	default	description
<code>a</code>	<code>point</code>	required	Start point.
<code>b</code>	<code>point</code>	required	End point.

RETURNS an open path of one segment.

EXAMPLE

```
let s = straight((0pt, 0pt), (60pt, 25pt))
mp-fig(
  draw(s, pen: broadnib(8pt, t: 1.5pt, angle: 20deg),
    fill: rgb("#1a4f8b")),
  // measured length
  mp-label(text(8pt)[length: #arclength(s)], (30pt,
    -10pt)),
  pad: 5pt)
```



polyline

```
polyline(pts, cycle: false)
```

Polygonal path through the points (straight segments).

PARAMETERS

name	type	default	description
<code>pts</code>	<code>array of points</code>	required	The vertices.
<code>cycle</code>	<code>bool</code>	<code>false</code>	Close the polygon.

RETURNS a path (one segment per side).

EXAMPLE

```
// regular pentagram
let pt(i) = (26pt * calc.cos(90deg + i * 144deg),
  26pt * calc.sin(90deg + i * 144deg))
let star = polyline(range(5).map(pt), cycle: true)
mp-fig(
  draw(star, pen: penrazor(3pt, angle: 45deg), fill:
    rgb("#9a3324")),
  pad: 4pt)
```



cubics

```
cubics(segments, cycle: false, y-down: false)
```

Path from a list of explicit cubic segments `((p0, c1, c2, p3), ...)`. The end of each segment is expected to be the start of the next one. With `y-down: true` the coordinates follow Typst's convention (y pointing down): the path is flipped and the pen angles are mirrored, which reproduces drawings written for the `nibst` package.

PARAMETERS

name	type	default	description
<code>segments</code>	<code>array</code>	<code>required</code>	Array of 4-point tuples <code>(p0, c1, c2, p3)</code> .
<code>cycle</code>	<code>bool</code>	<code>false</code>	Mark the path as closed.
<code>y-down</code>	<code>bool</code>	<code>false</code>	Interpret coordinates with y pointing down.

RETURNS a path.

EXAMPLE

```
// an S made of two cubics
let s = cubics((
  ((0pt, 0pt), (30pt, 0pt), (40pt, 20pt), (20pt,
    24pt)),
  ((20pt, 24pt), (0pt, 28pt), (10pt, 48pt), (40pt,
    48pt)),
))
nib-stroke(s, pad: 4pt, fill: rgb("#2b2b2b"),
  pen: nibpen(width: 7pt, thinness: 25%, angle: 40deg))
```



path-join

```
path-join(a, b)
```

Concatenates two open paths (the end of `a` is assumed to be the start of `b`; no check is made).

PARAMETERS

name	type	default	description
<code>a</code>	<code>path</code>	<code>required</code>	First path.
<code>b</code>	<code>path</code>	<code>required</code>	Second path.

RETURNS an open path.

EXAMPLE

```
let a = mp-path("(0,0)..(20,20)")
let b = mp-path("(20,20)..(40,0)..(60,20)")
mp-fig(
  draw(path-join(a, b), pen: pencircle(3pt), fill:
    rgb("#1a4f8b")),
  pad: 4pt)
```



path-join-all

```
path-join-all(parts, cycle: false)
```

Concatenates a whole list of paths, optionally marking the result as closed.

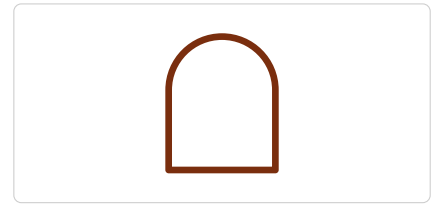
PARAMETERS

name	type	default	description
<code>parts</code>	<code>array of paths</code>	<code>required</code>	Paths to chain, in order.
<code>cycle</code>	<code>bool</code>	<code>false</code>	Close the result.

RETURNS a path.

EXAMPLE

```
// a rounded "tombstone": two lines + an arc
let up = straight((0pt, 0pt), (0pt, 30pt))
let arc = mp-path("(0,30){up}..(20,50){right}..{down}(40,30)")
let down = straight((40pt, 30pt), (40pt, 0pt))
let shape = path-join-all((up, arc, down), cycle: true)
mp-fig(draw(shape, pen: pencircle(2.5pt), fill:
rgb("#7a2e0e")), pad: 4pt)
```



path-close

```
path-close(p)
```

Marks an open path as cyclic: a straight join from the end to the start is understood.

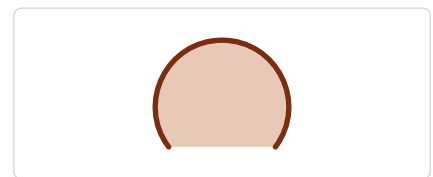
PARAMETERS

name	type	default	description
p	path	required	Open path.

RETURNS a closed path.

EXAMPLE

```
let open = mp-path("(0,0)..(20,40)..(40,0)")
mp-fig(
  // filled thanks to the implicit closing
  mp-fill(path-close(open), fill: rgb("#e8c9b8")),
  draw(open, pen: pencircle(2pt), fill: rgb("#7a2e0e")),
  pad: 4pt)
```



round-corners

```
round-corners(pts, r: 4pt, cycle: false)
```

Polygon whose corners are replaced by circular arcs of radius **r** (reduced automatically where the sides are too short).

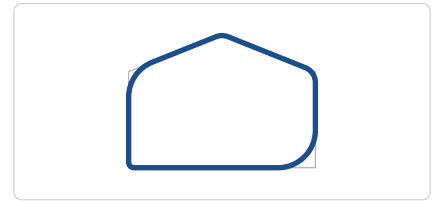
PARAMETERS

name	type	default	description
pts	array of points	required	Polygon vertices.
r	length or array	4pt	Radius, or one radius per vertex.
cycle	bool	false	Closed polygon (all corners rounded) or open (end points kept).

RETURNS a path.

EXAMPLE

```
let pts = ((0pt, 0pt), (70pt, 0pt), (70pt, 36pt), (35pt, 50pt), (0pt, 36pt))
mp-fig(
  mp-skeleton(polyline(pts, cycle: true), stroke: 0.4pt + gray),
  draw(
    round-corners(pts, r: (2pt, 14pt, 6pt, 6pt, 14pt), cycle: true),
    pen: pencircle(2pt), fill: rgb("#1a4f8b")),
  pad: 4pt)
```



join-smooth

```
join-smooth(a, b, tension: 1)
```

Joins the end of **a** to the start of **b** with a Hobby curve that leaves **a** and enters **b** along their tangents (no corner at the junction).

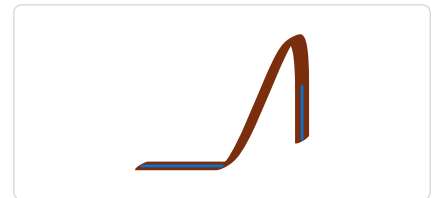
PARAMETERS

name	type	default	description
a	path	required	Open path to leave.
b	path	required	Open path to enter.
tension	number	1	Tension of the connecting curve.

RETURNS one open path **a** + connection + **b**.

EXAMPLE

```
let a = mp-path("(0,0){right}..(30,0)")
let b = mp-path("(60,30){down}..(60,10)")
mp-fig(
  draw(join-smooth(a, b), pen: broadnib(6pt, t: 1.4pt, angle: 30deg), fill: rgb("#7a2e0e")),
  draw(a, pen: pencircle(1pt), fill: blue), draw(b, pen: pencircle(1pt), fill: blue),
  pad: 4pt)
```



close-smooth

```
close-smooth(p, tension: 1)
```

Closes an open path with a smooth curve from its end back to its start, matching both tangents.

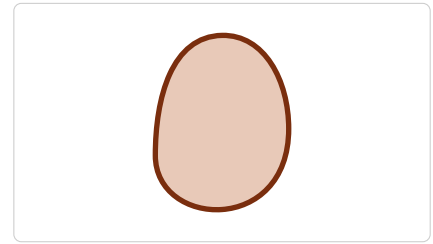
PARAMETERS

name	type	default	description
p	path	required	Open path.
tension	number	1	Tension of the closing curve.

RETURNS a closed path.

EXAMPLE

```
let p = mp-path("(0,0){up}..(25,45)..{down}(50,10)")
mp-fig(
  mp-fill(close-smooth(p), fill: rgb("#e8c9b8")),
  draw(close-smooth(p), pen: pencircle(2pt), fill:
    rgb("#7a2e0e")),
  pad: 4pt)
```



brace-path

```
brace-path(length, amplitude)
```

Bare path of a calligraphic brace from $(0,0)$ to $(length,0)$, bulging towards $y > 0$, its tip at height `amplitude`. See `delimiter` to draw it with a swelling pen.

PARAMETERS

name	type	default	description
<code>length</code>	<code>length</code>	required	Distance between the two ends.
<code>amplitude</code>	<code>length</code>	required	Height of the tip.

RETURNS an open path.

EXAMPLE

```
let b = brace-path(80pt, 10pt)
mp-fig(
  draw(b, pen: pencircle(1.6pt), fill: rgb("#9a3324")),
  mp-skeleton(straight((0pt, 0pt), (80pt, 0pt)), stroke:
    0.4pt + gray),
  pad: 4pt)
```



paren-path

```
paren-path(length, amplitude, straight: false)
```

Bare path of a parenthesis-like arc from $(0,0)$ to $(length,0)$, bulging towards $y > 0$. With `straight: true` the two curved ends are joined by a straight middle (a long bracket-parenthesis).

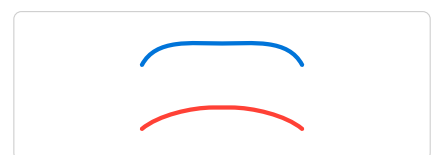
PARAMETERS

name	type	default	description
<code>length</code>	<code>length</code>	required	Distance between the ends.
<code>amplitude</code>	<code>length</code>	required	Bulge height.
<code>straight</code>	<code>bool</code>	<code>false</code>	Straight middle part.

RETURNS an open path.

EXAMPLE

```
mp-fig(
  draw(paren-path(60pt, 8pt), pen: pencircle(1.6pt),
    fill: blue),
  draw(shifted(paren-path(60pt, 8pt, straight: true),
    0pt, -24pt),
    pen: pencircle(1.6pt), fill: red),
  pad: 4pt)
```



Measuring and querying

Paths are parametrised by **time**: a path of `n` segments has times from `0` to `n`; time `2.5` is the middle of the third segment (as in MetaPost's `point 2.5 of p`). Functions that return positions give Typst lengths; angles are Typst angles.

path-length

`path-length(p)`

Number of Bézier segments of the path, i.e. its maximal time. (For the geometric length use `arclength`.)

PARAMETERS

name	type	default	description
<code>p</code>	<code>path</code>	required	Any path.

RETURNS an integer.

EXAMPLE

```
let p = mp-path("(0,0)..(30,30)..(60,0)..(90,30)")
let q = mp-path("(0,0)..(30,30)..(60,0)..cycle")
[open: #path-length(p) segments \
closed: #path-length(q) segments]
```

open: 3 segments
closed: 3 segments

arclength

`arclength(p)`

Total arc length of the path.

PARAMETERS

name	type	default	description
<code>p</code>	<code>path</code>	required	Any path.

RETURNS a length.

EXAMPLE

```
let half = mp-path("(0,0){right}..(40,40){left}")
let full = mp-path("(0,0)..(40,40)..(0,80)..
(-40,40)..cycle")
[arc: #calc.round(arclength(half).pt(), digits: 2) pt \
loop: #calc.round(arclength(full).pt(), digits: 2) pt]
```

arc: 81.29 pt
loop: 251.36 pt

arctime

`arctime(p, len)`

Time at which the path has covered the arc length `len` (MetaPost's `arctime len of p`). The result can then be given to `point-of`, `subpath`...

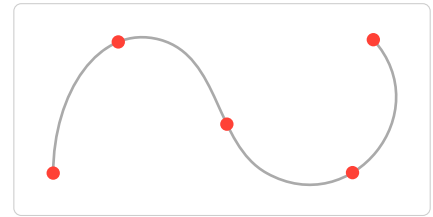
PARAMETERS

name	type	default	description
<code>p</code>	<code>path</code>	required	Path.
<code>len</code>	<code>length</code>	required	Distance travelled along the path.

RETURNS a number (time).

EXAMPLE

```
let p = mp-path("(0,0){up}..(40,50)..(80,0)..(120,50)")
let L = arclength(p)
// points every quarter of the length
// points à chaque quart de la longueur
let marks = (0, 0.25, 0.5, 0.75, 1).map(f =>
  point-of(p, arctime(p, L * f)))
mp-fig(
  draw(p, pen: pencircle(1pt), fill: gray),
  ..marks.map(m => mp-dot(m, pen: pencircle(5pt), fill:
    red)),
  pad: 4pt)
```



point-of

```
point-of(p, t)
```

Point of the path at time `t` (point `t` of `p`). On a cyclic path times wrap around.

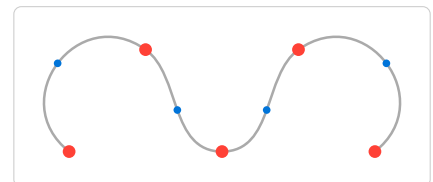
PARAMETERS

name	type	default	description
<code>p</code>	<code>path</code>	required	Path.
<code>t</code>	<code>number</code>	required	Time: segment index + fraction.

RETURNS a point `(x, y)` of lengths.

EXAMPLE

```
let p = mp-path("(0,0)..(30,40)..(60,0)..(90,40)..
(120,0)")
mp-fig(
  draw(p, pen: pencircle(1pt), fill: gray),
  // integer times = the nodes; halves = mid-segments
  // temps entiers = les nœuds ; demis = milieux
  ..range(9).map(i => mp-dot(point-of(p, i / 2),
    pen: pencircle(if calc.even(i) { 5pt } else
    { 3pt })),
    fill: if calc.even(i) { red } else { blue })),
  pad: 4pt)
```



direction-of

```
direction-of(p, t)
```

Tangent vector of the path at time `t` (not normalised; components in pt).

PARAMETERS

name	type	default	description
<code>p</code>	<code>path</code>	required	Path.
<code>t</code>	<code>number</code>	required	Time.

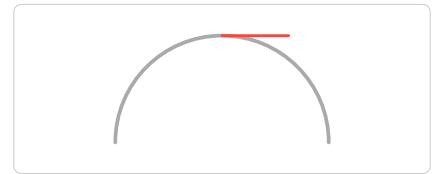
RETURNS an array `(dx, dy)` of numbers.

EXAMPLE

```

let p = mp-path("(0,0)..(40,40)..(80,0)")
let d = direction-of(p, 1)
let at = point-of(p, 1)
// draw the tangent at the top, scaled to a fixed length
// trace la tangente au sommet, à longueur fixe
let n = calc.sqrt(d.at(0) * d.at(0) + d.at(1) * d.at(1))
let e = (at.at(0) + d.at(0) / n * 25pt, at.at(1) +
d.at(1) / n * 25pt)
mp-fig(
  draw(p, pen: pencircle(1.4pt), fill: gray),
  draw(straight(at, e), pen: pencircle(1pt), fill: red),
  pad: 4pt)

```



direction-angle

`direction-angle(p, t)`

Angle of the tangent at time `t`, counter-clockwise from the x axis.

PARAMETERS

name	type	default	description
<code>p</code>	<code>path</code>	required	Path.
<code>t</code>	<code>number</code>	required	Time.

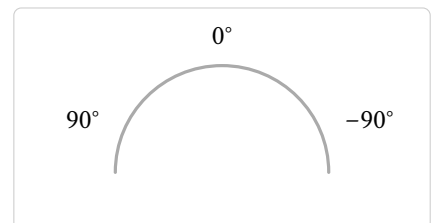
RETURNS an angle.

EXAMPLE

```

let p = mp-path("(0,0){up}..(40,40){right}..(80,0)
{down}")
let ang(t) = [#calc.round(direction-angle(p, t).deg())°]
mp-fig(
  draw(p, pen: pencircle(1.2pt), fill: gray),
  mp-label(ang(0), (-6pt, 20pt), align: right),
  mp-label(ang(1), (40pt, 48pt), align: bottom),
  mp-label(ang(2), (86pt, 20pt), align: left),
  pad: 14pt)

```



path-bbox

`path-bbox(p)`

Bounding box of the path (of the curve itself, not of any pen).

PARAMETERS

name	type	default	description
<code>p</code>	<code>path</code>	required	Path.

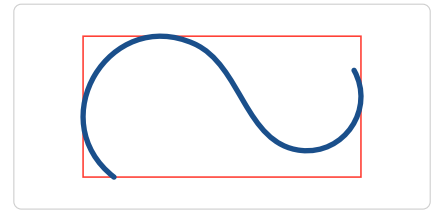
RETURNS a dictionary `(x0:, y0:, x1:, y1:)` of lengths.

EXAMPLE

```

let p = mp-path("(0,0)..(30,50)..(70,10)..(90,40)")
let b = path-bbox(p)
// rectangle drawn from the box
// rectangle tracé d'après la boîte
let r = polyline(((b.x0, b.y0), (b.x1, b.y0),
(b.x1, b.y1), (b.x0, b.y1)), cycle: true)
mp-fig(
  mp-skeleton(r, stroke: 0.6pt + red),
  draw(p, pen: pencircle(2pt), fill: rgb("#1a4f8b")),
  pad: 4pt)

```



closest-time

```
closest-time(p, pos)
```

Time of the point of the path nearest to `pos`, and the distance to it.

PARAMETERS

name	type	default	description
<code>p</code>	<code>path</code>	required	Path.
<code>pos</code>	<code>point</code>	required	Reference point <code>(x, y)</code> .

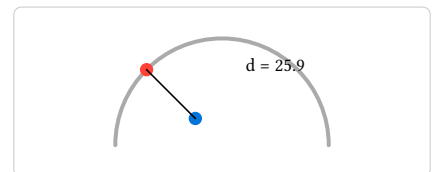
RETURNS an array `(t, distance)`.

EXAMPLE

```

let p = mp-path("(0,0)..(40,40)..(80,0)")
let q = (30pt, 10pt)
let (t, d) = closest-time(p, q)
mp-fig(
  draw(p, pen: pencircle(1.5pt), fill: gray),
  mp-dot(q, pen: pencircle(5pt), fill: blue),
  mp-dot(point-of(p, t), pen: pencircle(5pt), fill: red),
  draw(straight(q, point-of(p, t)), pen:
pencircle(0.6pt)),
  mp-label(text(7pt)[d = #calc.round(d.pt(), digits: 1)],
(60pt, 30pt)),
  pad: 4pt)

```



intersection-times

```
intersection-times(a, b)
```

All crossings of two paths, as pairs of times `(ta, tb)` sorted by `ta` (`intersectiontimes` of MetaPost, but returning **every** crossing).

PARAMETERS

name	type	default	description
<code>a</code>	<code>path</code>	required	First path.
<code>b</code>	<code>path</code>	required	Second path.

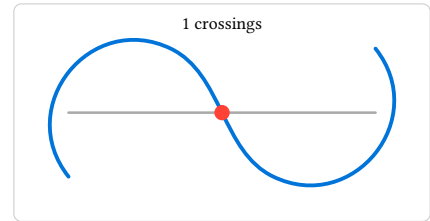
RETURNS an array of `(ta, tb)` pairs (empty if none).

EXAMPLE

```

let a = mp-path("(0,0)..(40,50)..(80,0)..(120,50)")
let b = mp-path("(0,25)--(120,25)")
let xs = intersection-times(a, b)
mp-fig(
  draw(a, pen: pencircle(1.5pt), fill: blue),
  draw(b, pen: pencircle(1pt), fill: gray),
  ..xs.map(x => mp-dot(point-of(a, x.at(0)),
    pen: pencircle(6pt), fill: red)),
  mp-label(text(7pt)[#xs.len() crossings], (60pt, 60pt)),
  pad: 6pt)

```



intersection-point

```
intersection-point(a, b)
```

The first crossing point of two paths (smallest time on `a`), or `none`.

PARAMETERS

name	type	default	description
<code>a</code>	<code>path</code>	required	First path.
<code>b</code>	<code>path</code>	required	Second path.

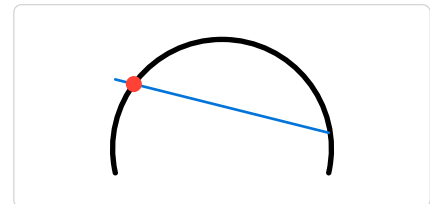
RETURNS a point, or `none`.

EXAMPLE

```

let a = mp-path("(0,0)..(40,50)..(80,0)")
let b = mp-path("(0,35)--(80,15)")
let x = intersection-point(a, b)
mp-fig(
  draw(a, pen: pencircle(2pt)),
  draw(b, pen: pencircle(1pt), fill: blue),
  mp-dot(x, pen: pencircle(6pt), fill: red),
  pad: 5pt)

```



self-intersections

```
self-intersections(p)
```

Self-crossings of a path: pairs of times `(t1, t2)` with `t1 < t2` at which the path passes through the same point.

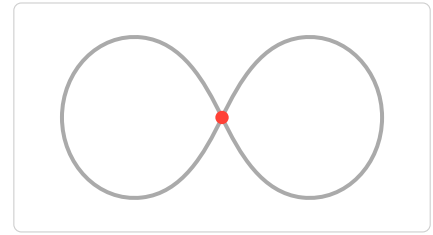
PARAMETERS

name	type	default	description
<code>p</code>	<code>path</code>	required	Path.

RETURNS an array of time pairs.

EXAMPLE

```
// a figure-eight loop
let p = mp-path("(0,0)..(30,30)..(60,0)..(30,-30)..
(0,0)..(-30,30)..(-60,0)..(-30,-30)..cycle")
let xs = self-intersections(p)
mp-fig(
  draw(p, pen: pencircle(1.5pt), fill: gray),
  ..xs.map(x => mp-dot(point-of(p, x.at(0)),
    pen: pencircle(5pt), fill: red)),
  pad: 5pt)
```



frame-at

```
frame-at(p, t)
```

Local frame of the path at time `t`: position and tangent direction. Give it to `in-frame` to place a drawing along the path.

PARAMETERS

name	type	default	description
<code>p</code>	<code>path</code>	required	Path.
<code>t</code>	<code>number</code>	required	Time.

RETURNS a dictionary `(pos: (x, y), angle: angle)`.

EXAMPLE

```
let p = mp-path("(0,0)..(40,40)..(80,0)")
let f = frame-at(p, 1)
[position: (#calc.round(f.pos.at(0).pt()),
#calc.round(f.pos.at(1).pt())) pt \
angle: #calc.round(f.angle.deg())°]
```

```
position: (40, 40) pt
angle: 0°
```

in-frame

```
in-frame(q, fr, scale: 1)
```

Places a path `q`, drawn in a **local** frame (x along the tangent, y to the left), into the frame `fr` returned by `frame-at`. Ideal for arrowheads, beads, thorns, hatching.

PARAMETERS

name	type	default	description
<code>q</code>	<code>path</code>	required	Path in local coordinates.
<code>fr</code>	<code>dictionary</code>	required	Frame from <code>frame-at</code> or <code>frames-along</code> .
<code>scale</code>	<code>number</code>	<code>1</code>	Uniform scale applied before placing.

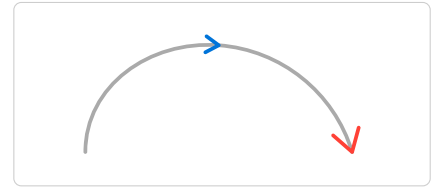
RETURNS a path.

EXAMPLE

```

let p = mp-path("(0,0){up}..(50,40)..(100,0)")
// an arrowhead drawn pointing along +x
// une pointe de flèche dessinée vers +x
let head = polyline((-8pt, 5pt), (0pt, 0pt), (-8pt, -5pt))
let end = frame-at(p, path-length(p))
mp-fig(
  draw(p, pen: pencircle(1.4pt), fill: gray),
  draw(in-frame(head, end), pen: pencircle(1.4pt), fill: red),
  draw(in-frame(head, frame-at(p, 1), scale: 0.6),
    pen: pencircle(1.4pt), fill: blue),
  pad: 5pt)

```



frames-along

```
frames-along(p, count: none, step: none, start: 0pt, end: 0pt)
```

Regularly spaced frames along a path (by arc length): `count` of them, evenly distributed (on a closed path, evenly around the loop), or one every `step`. Optional margins `start` and `end` are removed from the ends.

PARAMETERS

name	type	default	description
<code>p</code>	<code>path</code>	required	Path.
<code>count</code>	<code>integer or none</code>	<code>none</code>	Number of frames.
<code>step</code>	<code>length or none</code>	<code>none</code>	Distance between frames (alternative to <code>count</code>).
<code>start</code>	<code>length</code>	<code>0pt</code>	Arc length skipped at the start.
<code>end</code>	<code>length</code>	<code>0pt</code>	Arc length skipped at the end.

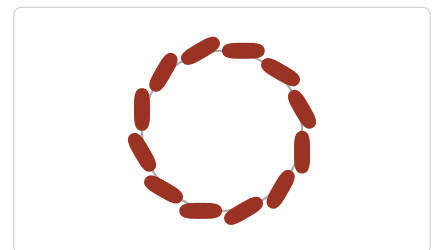
RETURNS an array of frames (`pos:`, `angle:`).

EXAMPLE

```

let ring = mp-path("(0,0)..(30,30)..(60,0)..
(30,-30)..cycle")
// 12 petals around a closed path, pointing outwards
// 12 pétales autour d'un chemin fermé, vers l'extérieur
let petal = mp-path("(0,0)..(8,3)..(16,0)..
(8,-3)..cycle")
mp-fig(
  draw(ring, pen: pencircle(0.8pt), fill: gray),
  ..frames-along(ring, count: 12).map(f =>
    mp-fill(in-frame(petal, f), fill: rgb("#9a3324"))),
  pad: 4pt)

```



Cutting and editing

These functions return **new** paths (paths are immutable). Those that cut a path into pieces return an array of paths.

subpath

```
subpath(p, t0, t1)
```

Part of the path between times `t0` and `t1` (`subpath(t0, t1)` of `p`). If `t1 < t0` the part is traversed backwards. On a cyclic path the part may wrap around the start; negative times are allowed.

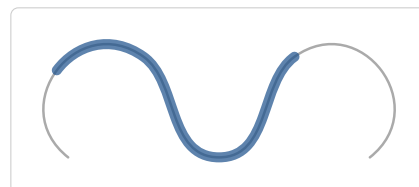
PARAMETERS

name	type	default	description
p	path	required	Path.
t0	number	required	Start time.
t1	number	required	End time.

RETURNS an open path.

EXAMPLE

```
let p = mp-path("(0,0)..(30,40)..(60,0)..(90,40)..(120,0)")
mp-fig(
  draw(p, pen: pencircle(1pt), fill: gray),
  // from the middle of segment 1 to the node 3
  // du milieu du segment 1 au nœud 3
  draw(subpath(p, 0.5, 3), pen: pencircle(4pt),
    fill: rgb("#1a4f8b").transparentize(30%)),
  pad: 5pt)
```



reverse

```
reverse(p)
```

The same path traversed in the opposite direction.

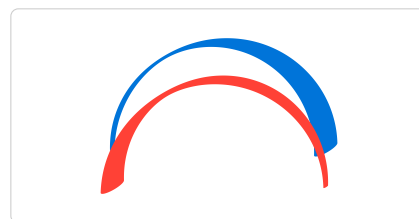
PARAMETERS

name	type	default	description
p	path	required	Path.

RETURNS a path.

EXAMPLE

```
let p = mp-path("(0,0){up}..(40,40)..(80,0)")
// both are drawn with a pen that thickens along the path
// les deux sont tracés avec une plume qui épaissit le
// long du chemin
let pen = nibpen((at: 0%, width: 2pt), (at: 100%, width: 10pt))
let r = shifted(reverse(p), 0pt, -14pt)
mp-fig(
  ..stroke-items(p, pen: pen, fill: blue),
  ..stroke-items(r, pen: pen, fill: red),
  pad: 4pt)
```



shorten

```
shorten(p, start: 0pt, end: 0pt)
```

Removes a length from the beginning and/or the end of an open path (e.g. to stop a line before an arrowhead).

PARAMETERS

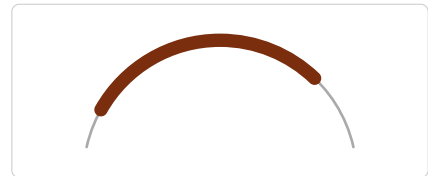
name	type	default	description
p	path	required	Open path.
start	length	0pt	Arc length removed at the start.

name	type	default	description
<code>end</code>	<code>length</code>	<code>0pt</code>	Arc length removed at the end.

RETURNS a path (error if nothing is left).

EXAMPLE

```
let p = mp-path("(0,0)..(50,40)..(100,0)")
mp-fig(
  draw(p, pen: pencircle(1pt), fill: gray),
  draw(shorten(p, start: 15pt, end: 30pt),
    pen: pencircle(5pt), fill: rgb("#7a2e0e")),
  pad: 4pt)
```



split-at

```
split-at(p, times)
```

Cuts a path at the given times. An open path cut at `n` times gives `n + 1` pieces; a closed path gives `n` pieces (the last one wraps around the start).

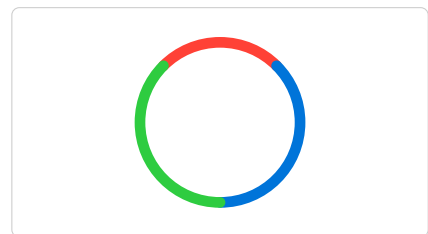
PARAMETERS

name	type	default	description
<code>p</code>	<code>path</code>	required	Path.
<code>times</code>	<code>array of numbers</code>	required	Cutting times.

RETURNS an array of paths.

EXAMPLE

```
let ring = mp-path("(0,0)..(30,30)..(60,0)..
(30,-30)..cycle")
let cols = (red, blue, green)
let parts = split-at(ring, (0.5, 1.5, 3))
mp-fig(
  ..parts.enumerate().map(((i, q)) => draw(
    q, pen: pencircle(4pt), fill: cols.at(i))),
  pad: 4pt)
```



remove-intervals

```
remove-intervals(p, ivs, min-piece: 0.05pt)
```

Removes arc-length intervals from a path and returns the remaining pieces. On closed paths the intervals may wrap around the start; the piece that straddles the seam is welded back together. Basis of over/under crossings.

PARAMETERS

name	type	default	description
<code>p</code>	<code>path</code>	required	Path.
<code>ivs</code>	<code>array of pairs</code>	required	Intervals <code>((a, b), ...)</code> of arc length (lengths).
<code>min-piece</code>	<code>length</code>	<code>0.05pt</code>	Pieces shorter than this are dropped.

RETURNS an array of paths.

EXAMPLE

```
let p = mp-path("(0,0)..(50,40)..(100,0)..(150,40)")
// remove two holes: [30, 45] and [90, 110]
// retire deux trous : [30, 45] et [90, 110]
let pieces = remove-intervals(p,
  ((30pt, 45pt), (90pt, 110pt)))
mp-fig(
  ..pieces.map(q => draw(q, pen: pencircle(4pt),
    fill: rgb("#1a4f8b"))),
  pad: 4pt)
```



gap-at

```
gap-at(p, times, half)
```

Opens a gap of half-length `half` (arc length) around each of the given times, and returns the pieces.

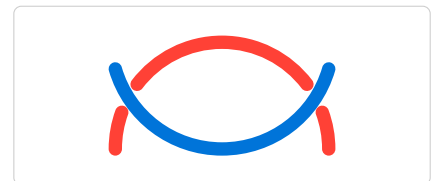
PARAMETERS

name	type	default	description
<code>p</code>	path	required	Path.
<code>times</code>	array of numbers	required	Times at which to cut.
<code>half</code>	length or array	required	Half-length of each gap (one value, or one per time).

RETURNS an array of paths.

EXAMPLE

```
// two strands crossing: the red one passes under
// deux brins qui se croisent : le rouge passe dessous
let a = mp-path("(0,0)..(40,40)..(80,0)")
let b = mp-path("(0,30)..(40,0)..(80,30)")
let ts = intersection-times(a, b).map(x => x.at(0))
mp-fig(
  draw(b, pen: pencircle(5pt), fill: blue),
  ..gap-at(a, ts, 6pt).map(q =>
    draw(q, pen: pencircle(5pt), fill: red)),
  pad: 4pt)
```



weld

```
weld(paths, tol: 0.05pt)
```

Joins consecutive paths of a list whenever the end of one lies within `tol` of the start of the next one.

PARAMETERS

name	type	default	description
<code>paths</code>	array of paths	required	Pieces, in order.
<code>tol</code>	length	<code>0.05pt</code>	Distance below which two ends are considered equal.

RETURNS an array of paths (fewer than given if some were welded).

EXAMPLE

```
let p = mp-path("(0,0)..(50,40)..(100,0)")
let (a, b, c) = split-at(p, (0.7, 1.4))
// a-b-c are three pieces; weld puts them back together
// a-b-c sont trois morceaux ; weld les remet bout à bout
let w = weld((a, b, c))
[pieces: 3 → #w.len()]
```

pieces: 3 → 1

fit-between

```
fit-between(p, a, b)
```

Applies to **p** the similarity (rotation + uniform scaling + translation) that maps its start to **a** and its end to **b**. Use it to stretch an ornament between two points.

PARAMETERS

name	type	default	description
p	path	required	Path whose end points differ.
a	point	required	Target of the start point.
b	point	required	Target of the end point.

RETURNS a path.

EXAMPLE

```
// a wave drawn once, stretched between several point
pairs
// une vague dessinée une fois, étirée entre plusieurs
paires de points
let wave = mp-path("(0,0)..(10,8)..(20,-8)..(30,0)")
let pairs = (((0pt, 0pt), (60pt, 0pt)), ((0pt, 10pt),
(50pt, 40pt)),
((0pt, -10pt), (30pt, -40pt)))
mp-fig(
  ..pairs.map(((a, b)) => draw(fit-between(wave, a, b),
    pen: pencircle(1.5pt), fill: rgb("#7a2e0e")),
    pad: 4pt)
```



offset

```
offset(p, d, tol: 0.02pt)
```

Parallel curve at signed distance **d** from the path (positive = to the **left** of the direction of travel). Pieces are joined by straight bevels; on a closed path the result is closed.

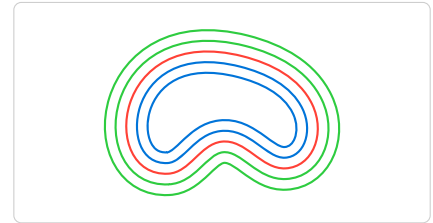
PARAMETERS

name	type	default	description
p	path	required	Path.
d	length	required	Signed distance.
tol	length	0.02pt	Approximation accuracy.

RETURNS a path.

EXAMPLE

```
let p = mp-path-pts(((0pt, 0pt), (20pt, 12pt), (44pt, 2pt),
(54pt, 26pt), (28pt, 44pt)), cycle: true)
mp-fig(
  ..range(-2, 3).map(k => stroke-items(
    offset(p, k * 4pt), pen: pencircle(0.8pt),
    fill: if k < 0 { green } else if k == 0 { red } else
{ blue })
  ).flatten(),
  pad: 3pt)
```



Transformations

Affine maps, as in MetaPost (`shifted`, `scaled`, `rotated`...). They act on the **path** — apply them before drawing. In MetaPost `p scaled 2 shifted (1,1)` reads left to right; here, nest the calls: `shifted(scaled(p, 2), 1pt, 1pt)`.

shifted

```
shifted(p, dx, dy)
```

Translates the path by `(dx, dy)`.

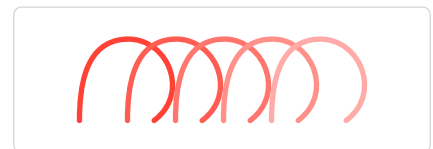
PARAMETERS

name	type	default	description
<code>p</code>	<code>path</code>	required	Path.
<code>dx</code>	<code>length</code>	required	Horizontal shift.
<code>dy</code>	<code>length</code>	required	Vertical shift (y up).

RETURNS a path.

EXAMPLE

```
let p = mp-path("(0,0){up}..(14,30)..(28,0)")
mp-fig(
  ..range(5).map(i => draw(shifted(p, i * 18pt, 0pt),
    pen: pencircle(2pt), fill: red.lighten(i * 14%))),
  pad: 4pt)
```



scaled

```
scaled(p, s)
```

Scales the path by the factor `s` about the origin.

PARAMETERS

name	type	default	description
<code>p</code>	<code>path</code>	required	Path.
<code>s</code>	<code>number</code>	required	Scale factor (negative = point reflection).

RETURNS a path.

EXAMPLE

```
let p = mp-path("(0,0){up}..(14,30)..(28,0)")
mp-fig(
  ..(0.4, 0.7, 1, 1.5).map(s => draw(scaled(p, s),
    pen: pencircle(1.5pt), fill: blue)),
  pad: 4pt)
```



xscaled

```
xscaled(p, s)
```

Scales horizontally only (x multiplied by s).

PARAMETERS

name	type	default	description
p	path	required	Path.
s	number	required	Horizontal factor.

RETURNS a path.

EXAMPLE

```
let p = mp-path("(0,0)..(15,15)..(30,0)..
(15,-15)..cycle")
mp-fig(
  draw(p, pen: pencircle(1.5pt), fill: gray),
  draw(shifted(xscaled(p, 2), 80pt, 0pt),
    pen: pencircle(1.5pt), fill: red),
  pad: 4pt)
```



yscaled

```
yscaled(p, s)
```

Scales vertically only (y multiplied by s). A negative factor flips the path upside down.

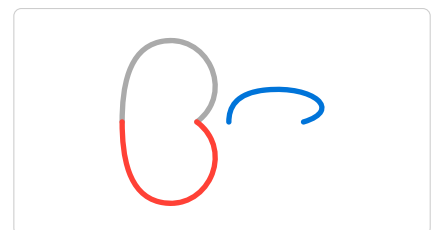
PARAMETERS

name	type	default	description
p	path	required	Path.
s	number	required	Vertical factor.

RETURNS a path.

EXAMPLE

```
let p = mp-path("(0,0){up}..(14,30)..(28,0)")
mp-fig(
  draw(p, pen: pencircle(2pt), fill: gray),
  draw(yscaled(p, -1), pen: pencircle(2pt), fill: red),
  draw(shifted(yscaled(p, 0.4), 40pt, 0pt),
    pen: pencircle(2pt), fill: blue),
  pad: 4pt)
```



slanted

```
slanted(p, s)
```

Shear: every point (x, y) becomes $(x + s \cdot y, y)$. Handy to slant lettering.

PARAMETERS

name	type	default	description
p	path	required	Path.
s	number	required	Slant ($0.25 \approx 14^\circ$).

RETURNS a path.

EXAMPLE

```
let p = mp-path("(0,0)--(0,40)--(25,40)--(25,0)--cycle")
mp-fig(
  ..(0, 0.25, 0.6).enumerate().map((i, s) =>
    draw(shifted(slanted(p, s), i * 45pt, 0pt),
      pen: pencircle(2pt), fill: rgb("#1a4f8b")),
    pad: 4pt)
```



rotated

```
rotated(p, a, about: (0pt, 0pt))
```

Rotates the path counter-clockwise by the angle **a** about the point **about**.

PARAMETERS

name	type	default	description
p	path	required	Path.
a	angle	required	Angle (a number is read as degrees).
about	point	(0pt, 0pt)	Centre of rotation.

RETURNS a path.

EXAMPLE

```
let p = mp-path("(10,0){up}..(24,30)..(38,0)")
// seven copies around the origin
// sept copies autour de l'origine
mp-fig(
  ..range(7).map(i => draw(rotated(p, i * 51.4deg),
    pen: broadnib(5pt, t: 1pt, angle: 30deg),
    fill: rgb("#7a2e0e").lighten(i * 9%))),
  pad: 3pt)
```



reflected

```
reflected(p, angle: 90deg)
```

Mirror image about the line through the origin whose direction is **angle** (90deg = vertical axis: left $\leftrightarrow$ right; 0deg = horizontal axis).

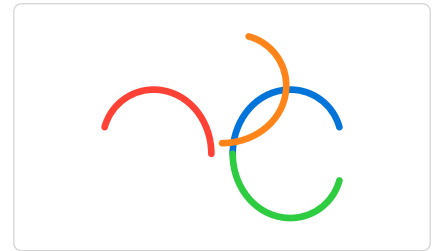
PARAMETERS

name	type	default	description
p	path	required	Path.
angle	angle	90deg	Direction of the mirror line.

RETURNS a path.

EXAMPLE

```
let p = mp-path("(4,0){up}..(24,24)..(44,10)")
let pen = pencircle(2.5pt)
mp-fig(
  draw(p, pen: pen, fill: blue),
  draw(reflected(p), pen: pen, fill: red),
  draw(reflected(p, angle: 0deg), pen: pen, fill: green),
  draw(reflected(p, angle: 45deg), pen: pen, fill:
orange),
  pad: 4pt)
```



transformed

```
transformed(p, a, b, c, d, e: 0pt, f: 0pt)
```

General affine map: $(x, y) \mapsto (a \cdot x + c \cdot y + e, b \cdot x + d \cdot y + f)$. All the other transformations are special cases (rotated is $a = d = \cos \theta$, $b = \sin \theta$, $c = -\sin \theta$).

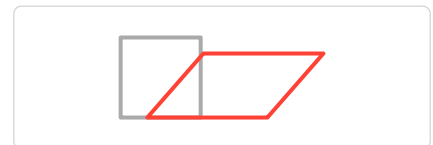
PARAMETERS

name	type	default	description
p	path	required	Path.
a, b, c, d	number	required	Coefficients of the 2×2 matrix.
e	length	0pt	Horizontal translation.
f	length	0pt	Vertical translation.

RETURNS a path.

EXAMPLE

```
let sq = mp-path("(0,0)--(30,0)--(30,30)--(0,30)--cycle")
// a unit square sheared and stretched into a
parallelogram
// un carré cisailé et étiré en parallélogramme
let para = transformed(sq, 1.5, 0, 0.7, 0.8, e: 10pt, f:
0pt)
mp-fig(
  draw(sq, pen: pencircle(1.5pt), fill: gray),
  draw(para, pen: pencircle(1.5pt), fill: red),
  pad: 4pt)
```



Pens

A **pen** is a plain dictionary describing the shape that is dragged along a path. Fixed pens (`penellipse`, `pencircle`, `broadnib`, `penpoly`, `pensquare`, `penrazor`) have a constant shape and work with `draw` and `stroke-items`; `nibpen` builds a calligraphic pen that can vary along the path and unlocks `dash`, `pressure` and `follow` in `stroke-items`.

Fixed pens

`penellipse`

```
penellipse(w, h, angle: 0deg)
```

Elliptical pen, like MetaPost's `pencircle xscaled w yscaled h rotated angle`. The stroke is thickest where the path is perpendicular to the long axis.

PARAMETERS

name	type	default	description
<code>w</code>	<code>length</code>	required	Full width, along the axis rotated by <code>angle</code> .
<code>h</code>	<code>length</code>	required	Full height, perpendicular to that axis.
<code>angle</code>	<code>angle</code>	<code>0deg</code>	Orientation of the <code>w</code> axis (counter-clockwise).

RETURNS a pen.

EXAMPLE

```
let p = mp-path("(0,0){up}..(15,28)..(30,0)..(45,28)")
let pens = (
  penellipse(10pt, 3pt),
  penellipse(10pt, 3pt, angle: 45deg),
  penellipse(3pt, 10pt),
)
grid(columns: 3, gutter: 6pt,
  ..pens.map(pn => nib-stroke(p, pen: pn, pad: 4pt)))
```



`pencircle`

```
pencircle(d)
```

Circular pen of diameter `d`: a stroke of constant width `d`.

PARAMETERS

name	type	default	description
<code>d</code>	<code>length</code>	required	Diameter.

RETURNS a pen.

EXAMPLE

```
let p = mp-path("(0,0)..(30,30)..(60,0)")
stack(dir: ltr, spacing: 8pt,
  ..(1pt, 3pt, 6pt, 10pt).map(d =>
    nib-stroke(p, pen: pencircle(d), pad: 2pt)))
```



broadnib

```
broadnib(w, t: auto, angle: 30deg)
```

Broad-edged calligraphic pen: a flattened ellipse. Thick strokes perpendicular to the nib, hairlines parallel to it.

PARAMETERS

name	type	default	description
<code>w</code>	<code>length</code>	required	Width of the nib.
<code>t</code>	<code>length or auto</code>	<code>auto</code>	Thickness of the nib (<code>auto</code> = <code>w / 5</code>).
<code>angle</code>	<code>angle</code>	<code>30deg</code>	Slant of the nib, counter-clockwise from horizontal.

RETURNS a pen.

EXAMPLE

```
// the letter "n" with three nib angles
// la lettre « n » avec trois angles de plume
let n = mp-path("(0,0){up}..(0,24)..(2,16){up}..(14,26)
{right}..(24,14){down}..(24,0)")
stack(dir: ltr, spacing: 10pt,
  ..(0deg, 30deg, 60deg).map(a =>
    nib-stroke(n, pen: broadnib(6pt, angle: a), pad:
      3pt)))
```



penpoly

```
penpoly(pts)
```

Convex polygonal pen. The points are given around the origin; the stroke is the exact union of the polygon dragged along the path.

PARAMETERS

name	type	default	description
<code>pts</code>	<code>array of points</code>	required	Vertices of a convex polygon (lengths).

RETURNS a pen.

EXAMPLE

```
let p = mp-path("(0,0)..(25,30)..(50,0)..(75,30)")
let tri = penpoly((-6pt, -4pt), (6pt, -4pt), (0pt,
  7pt))
nib-stroke(p, pen: tri, pad: 4pt)
```



pensquare

```
pensquare(s, angle: 0deg)
```

Square pen of side `s`, rotated by `angle`.

PARAMETERS

name	type	default	description
<code>s</code>	<code>length</code>	required	Side of the square.
<code>angle</code>	<code>angle</code>	<code>0deg</code>	Rotation (counter-clockwise).

RETURNS a pen.

EXAMPLE

```
let p = mp-path("(0,0)..(25,30)..(50,0)..(75,30)")
stack(dir: ltr, spacing: 8pt,
  ..(0deg, 20deg, 45deg).map(a =>
    nib-stroke(p, pen: pensquare(5pt, angle: a), pad:
      3pt)))
```



penrazor

```
penrazor(len, angle: 0deg)
```

Razor pen: a straight segment of length `len` at `angle`. Gives sharp, flat strokes whose width is zero when the path follows the razor.

PARAMETERS

name	type	default	description
<code>len</code>	<code>length</code>	required	Length of the segment.
<code>angle</code>	<code>angle</code>	<code>0deg</code>	Direction of the segment.

RETURNS a pen.

EXAMPLE

```
let p = mp-path("(0,0)..(25,30)..(50,0)..(75,30)")
stack(dir: ltr, spacing: 8pt,
  ..(0deg, 45deg, 90deg).map(a =>
    nib-stroke(p, pen: penrazor(10pt, angle: a), pad:
      3pt)))
```



The calligraphic pen and its options

nibpen

```
nibpen(..stops, width: 1pt, thinness: 25%, minor-width: none, angle: 30deg, follow:
false)
```

A broad-edged pen that may **change along the path**. Without stops its parameters are constant. A **stop** is a dictionary (`at:`, `width:`, `thinness:`, `minor-width:`, `angle:`); `at` is a ratio of the arc length (50%) or an absolute arc length (2cm). Keys missing from a stop take the defaults given to `nibpen`. Parameters are interpolated linearly between stops. With `follow: true` the angle is measured from the tangent of the path, so the nib turns with the stroke.

PARAMETERS

name	type	default	description
<code>..stops</code>	<code>dictionaries</code>	<code>()</code>	Optional stops (<code>at:</code> , <code>width:</code> , <code>thinness:</code> , <code>minor-width:</code> , <code>angle:</code>).
<code>width</code>	<code>length</code>	<code>1pt</code>	Default major axis (full width of the nib).
<code>thinness</code>	<code>ratio</code>	<code>25%</code>	Minor axis as a share of <code>width</code> .
<code>minor-width</code>	<code>length or none</code>	<code>none</code>	Minor axis as an absolute length (overrides <code>thinness</code>).
<code>angle</code>	<code>angle</code>	<code>30deg</code>	Default nib angle.
<code>follow</code>	<code>bool</code>	<code>false</code>	Angle relative to the tangent instead of fixed.

RETURNS a variable pen (use it with `stroke-items` / `nib-stroke`, not `draw`).

EXAMPLE

```
let p = mp-path("(0,0){up}..(25,45)..(60,10)..(95,50)")
let ink = rgb("#7a2e0e")
// 1. constant · 2. three stops · 3. follow the path
// 1. constante · 2. trois arrêts · 3. suit le chemin
let a = nibpen(width: 8pt, thinness: 20%, angle: 40deg)
let b = nibpen(
  (at: 0%, width: 3pt), (at: 50%, width: 14pt, thinness:
40%),
  (at: 100%, width: 3pt), angle: 40deg)
let c = nibpen(width: 9pt, thinness: 20%, angle: 90deg,
follow: true)
stack(dir: ttb, spacing: 6pt,
  ..(a, b, c).map(pn => nib-stroke(p, pen: pn, fill:
ink)))
```



dashes

```
dashes(..lengths, offset: 0pt, jitter: 0pt, seed: 0)
```

Dash pattern for `stroke-items(dash: ...)`: alternating dash and gap lengths along the path. `jitter` varies every length randomly (deterministically, from `seed`), which gives a hand-drawn look.

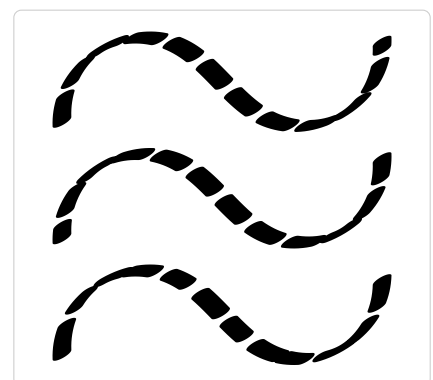
PARAMETERS

name	type	default	description
<code>..lengths</code>	<code>lengths</code>	required	Dash, gap, dash, gap... (repeated cyclically).
<code>offset</code>	<code>length</code>	<code>0pt</code>	Where the pattern starts.
<code>jitter</code>	<code>length</code>	<code>0pt</code>	Random variation $\pm$ <code>jitter</code> of every length.
<code>seed</code>	<code>integer</code>	<code>0</code>	Seed of the random sequence.

RETURNS a dash specification.

EXAMPLE

```
let p = mp-path("(0,0)..(40,30)..(80,0)..(120,30)")
let pen = nibpen(width: 8pt, thinness: 30%, angle: 30deg)
stack(dir: ttb, spacing: 4pt,
  nib-stroke(p, pen: pen, dash: dashes(10pt, 5pt)),
  nib-stroke(p, pen: pen, dash: dashes(10pt, 5pt, offset:
5pt)),
  nib-stroke(p, pen: pen,
    dash: dashes(10pt, 4pt, jitter: 3pt, seed: 7)))
```



pressure

```
pressure(minimum-width: 0.2pt, period: 2pt, seed: 0)
```

Simulates a pen being lifted: wherever the minor axis of the pen is below `minimum-width`, the stroke breaks up with a probability that grows as the pen gets thinner (one random draw per `period` of arc length). Use it with a pen that thins out.

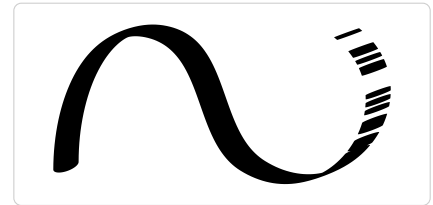
PARAMETERS

name	type	default	description
<code>minimum-width</code>	<code>length</code>	<code>0.2pt</code>	Minor axis below which the stroke starts to break up.
<code>period</code>	<code>length</code>	<code>2pt</code>	Length of each random draw.
<code>seed</code>	<code>integer</code>	<code>0</code>	Seed of the random sequence.

RETURNS a pressure specification.

EXAMPLE

```
let p = mp-path("(0,0){up}..(35,50)..(70,0)..(105,50)")
// the pen thins from 40% to 2% of its width
// la plume s'amincit de 40 % à 2 % de sa largeur
let pen = nibpen((at: 0%, width: 10pt, thinness: 40%),
  (at: 100%, width: 10pt, thinness: 2%), angle: 20deg)
nib-stroke(p, pen: pen,
  pressure: pressure(minimum-width: 1.2pt, period: 1.5pt,
    seed: 3))
```



Drawing

Drawing functions return **drawables**: opaque objects that you pass to `mp-fig`, which lays them out in a y-up coordinate system and returns an ordinary Typst `box`. `nib-stroke` is the shortcut for one calligraphic stroke in its own box.

Strokes

draw

```
draw(p, pen: none, pens: none, fill: black, tol: 0.01pt, fit: true, exact: false,
outline: none)
```

Draws path `p` with a pen, like MetaPost's `pickup pen`; `draw p`. The result is the exact **filled shape** swept by the pen. Pass either a fixed pen (`pen`) or, for a variable pen at the lowest level, `pens`: one `(w, h, angle)` per path node. For `nibpen`, dashes, pressure... use `stroke-items`.

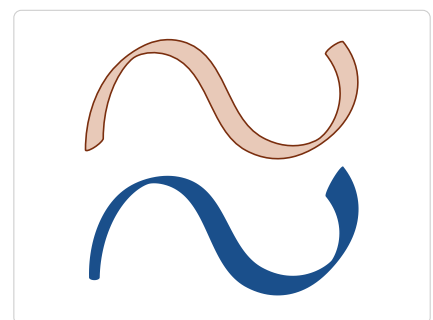
PARAMETERS

name	type	default	description
<code>p</code>	<code>path</code>	required	Path to draw.
<code>pen</code>	<code>pen</code>	<code>none</code>	Fixed pen (<code>penellipse</code> , <code>pencircle</code> , <code>broadnib</code> , <code>penpoly</code> , <code>pensquare</code> , <code>penrazor</code>).
<code>pens</code>	<code>array or function</code>	<code>none</code>	Variable pen: one <code>(w, h, angle)</code> per node (<code>segments + 1</code> , or <code>segments</code> on a closed path), or a function of the node index.
<code>fill</code>	<code>color</code>	<code>black</code>	Fill of the shape.
<code>tol</code>	<code>length</code>	<code>0.01pt</code>	Geometric tolerance of the envelope.
<code>fit</code>	<code>bool</code>	<code>true</code>	Fit Bézier curves to the outline of variable/polygonal pens (fewer nodes).
<code>exact</code>	<code>bool</code>	<code>false</code>	Force the sweep by union of convex hulls instead of the exact envelope.
<code>outline</code>	<code>stroke or none</code>	<code>none</code>	Typst stroke drawn around the shape.

RETURNS a drawable.

EXAMPLE

```
let p = mp-path("(0,0){up}..(30,36)..(60,0)..(90,36)")
mp-fig(
  // fixed pen with a Typst outline
  // plume fixe avec un contour Typst
  draw(p, pen: broadnib(8pt, t: 2pt, angle: 35deg),
    fill: rgb("#e8c9b8"), outline: 0.6pt +
    rgb("#7a2e0e")),
  // variable pen: one (w, h, angle) per node (4 nodes)
  // plume variable : un (w, h, angle) par nœud (4 nœuds)
  draw(shifted(p, 0pt, -50pt),
    pens: i => (4pt + i * 3pt, 2pt, 20deg * i),
    fill: rgb("#1a4f8b"),
    pad: 4pt)
```



stroke-items

```
stroke-items(p, pen: none, pens: none, fill: black, dash: none, pressure: none, follow:
auto, refine: auto, overlap: "union", layer: 4pt, outline: none, tol: 0.01pt, exact:
false, fit: true, debug: false)
```

The all-in-one calligraphic stroke. Returns an **array** of drawables (spread it into `mp-fig` with `..`). Accepts every kind of pen; `nibpen`-specific features need an elliptical pen: dashes, pressure, `overlap`: "layered" and `debug`.

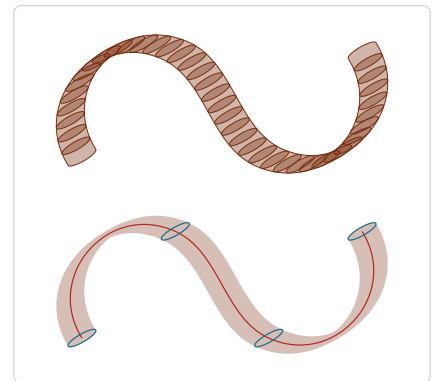
PARAMETERS

name	type	default	description
<code>p</code>	<code>path</code>	required	Path to draw.
<code>pen</code>	<code>pen</code>	<code>none</code>	Any pen, including <code>nibpen(...)</code> .
<code>pens</code>	<code>array or function</code>	<code>none</code>	Legacy per-node pens (as in <code>draw</code>); cannot be combined with dash, pressure, layered or debug.
<code>fill</code>	<code>color</code>	<code>black</code>	Fill of the shapes (translucent colours work with <code>layered</code>).
<code>dash</code>	<code>dashes</code>	<code>none</code>	Dash pattern from <code>dashes(...)</code> .
<code>pressure</code>	<code>pressure</code>	<code>none</code>	Pen-lifting from <code>pressure(...)</code> .
<code>follow</code>	<code>bool or auto</code>	<code>auto</code>	Overrides <code>nibpen(follow:)</code> .
<code>refine</code>	<code>integer or auto</code>	<code>auto</code>	Subdivision of each segment for varying pens (<code>auto</code> = 4 when the pen varies, else 1).
<code>overlap</code>	<code>string</code>	<code>"union"</code>	"union": one merged shape; "layered": pieces of length <code>layer</code> stacked as separate shapes.
<code>layer</code>	<code>length</code>	<code>4pt</code>	Length of a piece in <code>layered</code> mode.
<code>outline</code>	<code>stroke or none</code>	<code>none</code>	Typst stroke around the shapes.
<code>tol, exact, fit</code>	<code>—</code>	<code>0.01pt, false, true</code>	As in <code>draw</code> .
<code>debug</code>	<code>bool</code>	<code>false</code>	Also draw the skeleton (red) and some pen positions (cyan).

RETURNS an array of drawables.

EXAMPLE

```
let p = mp-path("(0,0)..(35,40)..(70,0)..(105,40)")
let ink = rgb("#7a2e0e")
let pen = nibpen(width: 12pt, thinness: 25%, angle: 30deg)
stack(dir: ttb, spacing: 8pt,
  // layered, translucent, with outline
  // empilé, translucide, avec contour
  mp-fig(..stroke-items(p, pen: pen, overlap: "layered",
    layer: 5pt, fill: ink.transparentize(65%),
    outline: 0.3pt + ink), pad: 4pt),
  // construction view
  // vue de construction
  mp-fig(..stroke-items(p, pen: pen, debug: true,
    fill: ink.transparentize(70%)), pad: 4pt))
```



nib-stroke

```
nib-stroke(p, pad: 1pt, ..args)
```

Like `stroke-items`, but returns a ready-to-use box (an `mp-fig` fitted to the stroke plus `pad`). Every other argument is passed on to `stroke-items`.

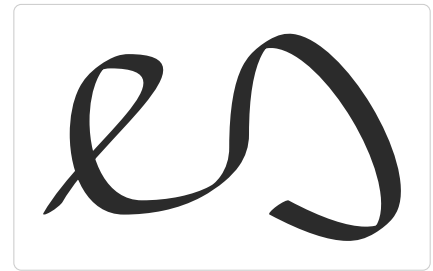
PARAMETERS

name	type	default	description
<code>p</code>	<code>path</code>	required	Path to draw.
<code>pad</code>	<code>length</code>	<code>1pt</code>	Margin around the stroke.
<code>..args</code>	<code>—</code>	required	Any argument of <code>stroke-items</code> (<code>pen</code> , <code>fill</code> , <code>dash</code> , ...).

RETURNS a box (baseline on the `y = 0` line when the stroke crosses it).

EXAMPLE

```
let p = mp-path("(0,0){dir 60}..(25,55){left}..(10,30)
{down}..(30,0){right}..(70,25){up}..(85,0)")
nib-stroke(p, pad: 4pt, fill: rgb("#2b2b2b"),
  pen: broadnib(9pt, t: 1.8pt, angle: 35deg))
```



Fills, dots, labels

mp-fill

```
mp-fill(p, fill: black)
```

Fills the area enclosed by a closed path (`fill p`).

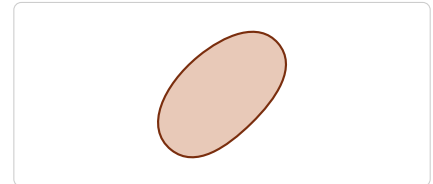
PARAMETERS

name	type	default	description
<code>p</code>	<code>path</code>	required	Closed path (an open path is closed by a straight line).
<code>fill</code>	<code>color</code>	<code>black</code>	Fill colour.

RETURNS a drawable.

EXAMPLE

```
let blob = mp-path("(0,0)..(30,10)..(40,40)..
(10,35)..cycle")
mp-fig(
  mp-fill(blob, fill: rgb("#e8c9b8")),
  mp-skeleton(blob, stroke: 0.8pt + rgb("#7a2e0e")),
  pad: 4pt)
```



mp-dot

```
mp-dot(pos, pen: pencircle(2pt), fill: black)
```

A dot: the pen shape drawn at one position (`drawdot`).

PARAMETERS

name	type	default	description
<code>pos</code>	<code>point</code>	required	Position (<code>x</code> , <code>y</code>).
<code>pen</code>	<code>pen</code>	<code>pencircle(2pt)</code>	Pen whose shape is the dot (a round, elliptical, square... dot).
<code>fill</code>	<code>color</code>	<code>black</code>	Colour.

RETURNS a drawable.

EXAMPLE

```
mp-fig(
  mp-dot((0pt, 0pt), pen: pencircle(8pt), fill: red),
  mp-dot((20pt, 0pt), pen: pensquare(7pt, angle: 20deg),
  fill: blue),
  mp-dot((40pt, 0pt), pen: penellipse(12pt, 5pt, angle:
  30deg)),
  mp-dot((60pt, 0pt), pen: penpoly((( -5pt, -4pt), (5pt,
  -4pt), (0pt, 6pt))), fill: green),
  pad: 4pt)
```



mp-label

```
mp-label(body, pos, align: center + horizon)
```

A Typst content placed at a position, aligned like MetaPost's `label.top/.lft...` `align` says which point of the content sits at `pos`. Labels do not count in the automatic bounding box of `mp-fig` (use `pad`).

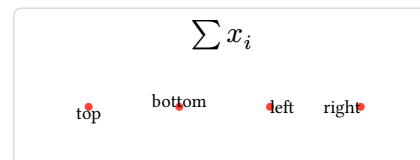
PARAMETERS

name	type	default	description
<code>body</code>	<code>content</code>	required	Any content: text, maths, an image...
<code>pos</code>	<code>point</code>	required	Position (x, y).
<code>align</code>	<code>alignment</code>	<code>center + horizon</code>	Anchor of the content: <code>top</code> , <code>bottom</code> , <code>left</code> , <code>right</code> , <code>center</code> , <code>horizon</code> and their combinations.

RETURNS a drawable.

EXAMPLE

```
// each dot is the anchor `pos`; the label hangs from it
// chaque point est l'ancree `pos` ; l'étiquette s'y
// accroche
let anchors = (top, bottom, left, right)
mp-fig(
  ..anchors.enumerate().map((i, a)) => {
    let o = (i * 34pt, 0pt)
    mp-group(
      mp-dot(o, pen: pencircle(3pt), fill: red),
      mp-label(text(7pt, repr(a)), o, align: a)
    ),
    mp-label(text(11pt)[$sum x_i$], (50pt, 28pt)),
    width: 140pt, height: 46pt, origin: (20pt, 16pt))
```



mp-skeleton

```
mp-skeleton(p, stroke: 0.4pt + red)
```

Draws the bare skeleton of a path with a Typst `stroke` (handy to check a path before choosing a pen).

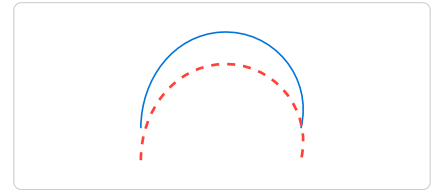
PARAMETERS

name	type	default	description
<code>p</code>	<code>path</code>	required	Path.
<code>stroke</code>	<code>stroke</code>	<code>0.4pt + red</code>	Any Typst stroke (dashed, coloured...).

RETURNS a drawable.

EXAMPLE

```
let p = mp-path("(0,0){up}..(30,36)..(60,0)")
mp-fig(
  mp-skeleton(p, stroke: 0.6pt + blue),
  mp-skeleton(shifted(p, 0pt, -12pt),
    stroke: (paint: red, thickness: 1pt, dash:
      "dashed")),
  pad: 4pt)
```



mp-group

```
mp-group(..items)
```

Groups several drawables into one (so a function can return a single object). `mp-fig` also accepts arrays of drawables directly, so grouping is optional.

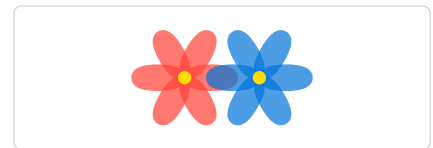
PARAMETERS

name	type	default	description
<code>..items</code>	<code>drawables</code>	required	Objects to group (arrays are flattened).

RETURNS a drawable.

EXAMPLE

```
// a reusable "flower": one object made of several
// une « fleur » réutilisable : un objet fait de
// plusieurs
let petal = mp-path("(0,0)..(10,5)..(20,0)..
(10,-5)..cycle")
let flower(c, dx) = mp-group(
  ..range(6).map(i => mp-fill(
    shifted(rotated(petal, i * 60deg), dx, 0pt),
    fill: c.transparentize(30%))),
  mp-dot((dx, 0pt), pen: pencircle(5pt), fill: yellow))
mp-fig(flower(red, 0pt), flower(blue, 28pt), pad: 2pt)
```



Figures

mp-fig

```
mp-fig(..items, width: auto, height: auto, origin: (0pt, 0pt), pad: 1pt, baseline: auto,
  grid: none, frame: none, clip: false)
```

Collects drawables in a y-up coordinate system (like `beginfig ... endfig`) and returns a `box`. With `auto` sizes the box is fitted to the content plus `pad`; with explicit sizes `origin` is the position of the origin counted from the bottom-left corner. The box can be used inline in running text: its baseline is the line `y = 0` unless `baseline` is given.

PARAMETERS

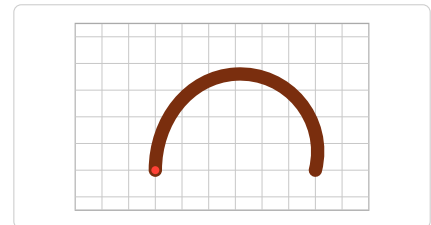
name	type	default	description
<code>..items</code>	<code>drawables</code>	required	Objects to draw, in order (later ones on top). Arrays are flattened: <code>..stroke-items(...)</code> or <code>stroke-items(...)</code> both work.
<code>width</code>	<code>length or auto</code>	<code>auto</code>	Width of the box (<code>auto</code> = fit the content).
<code>height</code>	<code>length or auto</code>	<code>auto</code>	Height of the box.
<code>origin</code>	<code>point</code>	<code>(0pt, 0pt)</code>	Position of the origin from the bottom-left corner (explicit sizes only).
<code>pad</code>	<code>length</code>	<code>1pt</code>	Margin added around the content when a size is <code>auto</code> .

name	type	default	description
baseline	length or auto	auto	Baseline offset from the bottom of the box.
grid	length or none	none	Draws a light grid with this spacing (for design work).
frame	stroke or none	none	Border stroke around the box.
clip	bool	false	Clip drawing that leaves the box.

RETURNS a `box`.

EXAMPLE

```
let p = mp-path("(0,0){up}..(30,36)..(60,0)")
// explicit size, origin in the middle, grid, frame
// taille explicite, origine au milieu, grille, cadre
mp-fig(
  draw(p, pen: pencircle(5pt), fill: rgb("#7a2e0e")),
  mp-dot((0pt, 0pt), pen: pencircle(3pt), fill: red),
  width: 110pt, height: 70pt, origin: (30pt, 15pt),
  grid: 10pt, frame: 0.5pt + gray, clip: true)
```



Knots, ornaments and decorations

This layer (`art.typ`, re-exported by the package) turns the geometry engine into ready-made ornaments. The ideas follow the LaTeX packages `spath3`, `knots` and `calligraphy`; the implementation is independent. Like everything else, these functions return arrays of drawables (spread them into `mp-fig` with `..`).

Knots and links

knot

```
knot(strands, style: "gap", rule: "alternate", width: 4pt, fill: black, outline: none,
outline-fill: black, gap: auto, flips: (), draft: false, draft-size: 7pt)
```

Interlaced strands: knots, links, braids, Celtic work. Pass one path or an array of paths (open or closed). Every crossing — between strands or inside one strand — is detected automatically and numbered in order of first encounter. `style: "gap"` interrupts the under-strand (classic knot diagram); `style: "weave"` draws each strand as a ribbon with an outline and redraws the over-strand on top at each crossing (Celtic interlace; needs `outline`).

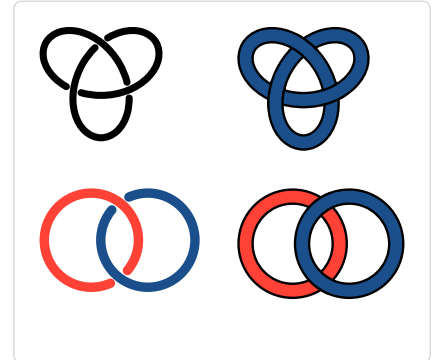
PARAMETERS

name	type	default	description
<code>strands</code>	path or array of paths	required	The strands.
<code>style</code>	string	"gap"	"gap" or "weave" (without <code>outline</code> , "weave" falls back to "gap").
<code>rule</code>	string	"alternate"	"alternate": alternating diagram (over, under, over... along every strand). "first": the strand of lower index passes over (within one strand, the later passage).
<code>width</code>	length or array	4pt	Width of the strands (one value, or one per strand).
<code>fill</code>	color or array	black	Colour of the strands (one value, or one per strand).
<code>outline</code>	length or none	none	Width of the outline on each side of a ribbon.
<code>outline-fill</code>	color or array	black	Colour of the outline.
<code>gap</code>	length or auto	auto	Clearance on each side of a gap, "gap" style (<code>auto</code> = $0.6 \times \text{width}$).
<code>flips</code>	array of integers	()	Numbers of the crossings to invert.
<code>draft</code>	bool	false	Show the strand skeletons and the crossing numbers (to pick <code>flips</code>).
<code>draft-size</code>	length	7pt	Size of the numbers in draft mode.

RETURNS an array of drawables.

EXAMPLE

```
// a trefoil: 24 points of a parametric curve
// un trèfle : 24 points d'une courbe paramétrique
let pt(i) = {
  let t = i * 15deg
  (9pt * (calc.sin(t) + 2 * calc.sin(2 * t)),
   9pt * (calc.cos(t) - 2 * calc.cos(2 * t)))
}
let tre = mp-path-pts(range(24).map(pt), cycle: true)
// a link of two circles
// un lien de deux cercles
let c1 = mp-path("(0,0)..(20,20)..(40,0)..
(20,-20)..cycle")
let c2 = shifted(c1, 24pt, 0pt)
let blue = rgb("#1a4f8b")
grid(columns: 2, gutter: 10pt,
  mp-fig(..knot(tre, width: 3pt), pad: 3pt),
  mp-fig(..knot(tre, style: "weave", width: 5pt, fill:
blue,
  outline: 1pt), pad: 3pt),
  mp-fig(..knot((c1, c2), width: 4pt, fill: (red, blue)),
  pad: 3pt),
  mp-fig(..knot((c1, c2), style: "weave", width: 5pt,
  fill: (red, blue), outline: 1pt, flips: (1,)), pad:
3pt))
```



crossings

```
crossings(strands)
```

The raw data behind `knot`: every crossing of a set of strands, including self-intersections of each strand.

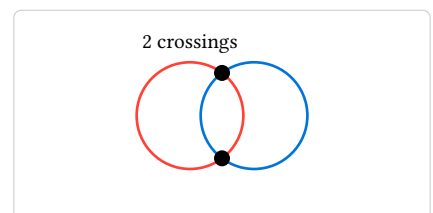
PARAMETERS

name	type	default	description
strands	path or array of paths	required	The strands.

RETURNS an array of (a: (strand, t), b: (strand, t)): strand index and time of the two passages.

EXAMPLE

```
let c1 = mp-path("(0,0)..(20,20)..(40,0)..
(20,-20)..cycle")
let c2 = shifted(c1, 24pt, 0pt)
let cs = crossings((c1, c2))
mp-fig(
  draw(c1, pen: pencircle(1pt), fill: red),
  draw(c2, pen: pencircle(1pt), fill: blue),
  // the point where the first strand passes through each
crossing
  // le point où le premier brin traverse chaque
croisement
  ..cs.map(c => mp-dot(point-of(c1, c.a.at(1)),
  pen: pencircle(6pt), fill: black)),
  mp-label(text(8pt)[#cs.len() crossings], (20pt, 28pt)),
  pad: 12pt)
```



Calligraphic decorations

copperplate

```
copperplate(p, light: 0.4pt, heavy: 2.4pt, slant: 90deg, sharpness: 1.0, taper: "both",  
taper-length: 6pt, step: 1.5pt, fill: black, outline: none)
```

Stroke with a **pointed pen** (copperplate, roundhand): a hairline on the upstrokes, a shade on the downstrokes. The width follows the direction of the path relative to the shading axis `slant`. Returns an array of drawables.

PARAMETERS

name	type	default	description
<code>p</code>	<code>path</code>	required	Path of the stroke.
<code>light</code>	<code>length</code>	<code>0.4pt</code>	Hairline width.
<code>heavy</code>	<code>length</code>	<code>2.4pt</code>	Width of the shade.
<code>slant</code>	<code>angle</code>	<code>90deg</code>	Direction of the shading axis (<code>90deg</code> = vertical; smaller = italic).
<code>sharpness</code>	<code>number</code>	<code>1.0</code>	> 1 confines the shade to strokes closer to the axis.
<code>taper</code>	<code>string or none</code>	<code>"both"</code>	Hairline entry/exit: <code>none</code> , <code>"start"</code> , <code>"end"</code> , <code>"both"</code> .
<code>taper-length</code>	<code>length</code>	<code>6pt</code>	Length of the tapered ends.
<code>step</code>	<code>length</code>	<code>1.5pt</code>	Sampling distance (smaller = smoother, slower).
<code>fill</code>	<code>color</code>	<code>black</code>	Ink colour.
<code>outline</code>	<code>stroke or none</code>	<code>none</code>	Typst stroke around the shape.

RETURNS an array of drawables.

EXAMPLE

```
let p = mp-path("(0,0)..(10,22){up}..(24,40)..(30,20)..  
(18,0)..(14,14)..(30,26)..(52,8)..(66,26)")  
let ink = rgb("#1d2a44")  
stack(dir: ttb, spacing: 6pt,  
  mp-fig(..copperplate(p, light: 0.5pt, heavy: 3pt,  
    slant: 70deg, sharpness: 1.4, fill: ink), pad: 3pt),  
  // no taper, heavier shade  
  // sans effilage, plein plus lourd  
  mp-fig(..copperplate(p, light: 0.5pt, heavy: 4.5pt,  
    taper: none, fill: ink), pad: 3pt))
```



prongs

```
prongs(p, offsets, width: 0.6pt, fill: black, tol: 0.02pt)
```

A multi-tine nib (split nib, engraver's pen): one thin line per offset, parallel to the path.

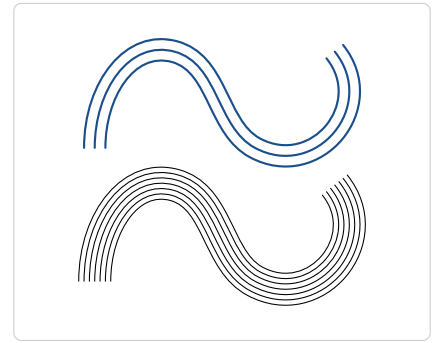
PARAMETERS

name	type	default	description
<code>p</code>	<code>path</code>	required	Path.
<code>offsets</code>	<code>array of lengths</code>	required	Offsets of the tines (positive = left of the direction of travel).
<code>width</code>	<code>length</code>	<code>0.6pt</code>	Width of each line.
<code>fill</code>	<code>color</code>	<code>black</code>	Colour.
<code>tol</code>	<code>length</code>	<code>0.02pt</code>	Accuracy of the offset curves.

RETURNS an array of drawables.

EXAMPLE

```
let p = mp-path("(0,0){up}..(30,36)..(60,0)..(90,36)")
mp-fig(
  ..prongs(p, (-4pt, 0pt, 4pt), width: 0.8pt,
    fill: rgb("#1a4f8b")),
  ..prongs(shifted(p, 0pt, -50pt),
    range(-3, 4).map(i => i * 2pt), width: 0.4pt),
  pad: 6pt)
```



delimiter

```
delimiter(a, b, kind: "brace", amplitude: 6pt, flip: false, light: 0.4pt, heavy: 1.8pt,
  fill: black)
```

A calligraphic brace or parenthesis between two points, bulging to the left of $a \rightarrow b$; the pen swells where the curve turns. For the bare paths, see `brace-path` and `paren-path`.

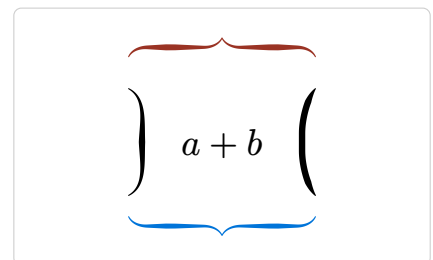
PARAMETERS

name	type	default	description
<code>a</code>	point	required	Start point.
<code>b</code>	point	required	End point.
<code>kind</code>	string	<code>"brace"</code>	<code>"brace"</code> , <code>"paren"</code> (curved) or <code>"paren-straight"</code> (straight middle).
<code>amplitude</code>	length	<code>6pt</code>	How far the delimiter bulges.
<code>flip</code>	bool	<code>false</code>	Bulge to the right of $a \rightarrow b$ instead.
<code>light</code>	length	<code>0.4pt</code>	Pen width at the ends.
<code>heavy</code>	length	<code>1.8pt</code>	Pen width where the curve swells.
<code>fill</code>	color	<code>black</code>	Ink colour.

RETURNS an array of drawables.

EXAMPLE

```
mp-fig(
  ..delimiter((0pt, 0pt), (70pt, 0pt), amplitude: 7pt,
    heavy: 2.4pt, fill: rgb("#9a3324")),
  ..delimiter((0pt, -12pt), (0pt, -52pt), kind: "paren",
    amplitude: 5pt, heavy: 2.4pt),
  ..delimiter((70pt, -52pt), (70pt, -12pt),
    kind: "paren-straight", amplitude: 5pt, heavy:
    2.4pt),
  // flip: the brace bulges downwards
  // flip : l'accolade bombe vers le bas
  ..delimiter((0pt, -60pt), (70pt, -60pt), flip: true,
    amplitude: 7pt, heavy: 2pt, fill: blue),
  mp-label(text(14pt)[$a+b$], (35pt, -32pt)),
  pad: 4pt)
```



Version

nib-version

```
nib-version
```

The package version, as a string. Useful in headers and bug reports.

RETURNS a string, e.g. "0.3.0".

EXAMPLE

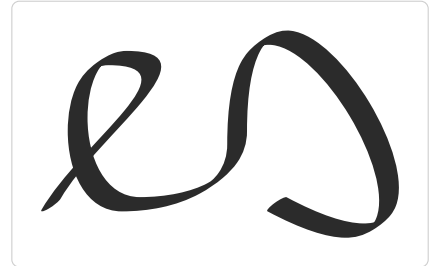
```
[nibart version: #nib-version]
```

```
nibart version: 0.3.0
```

Recipes

Handwriting-like loop with a broad nib

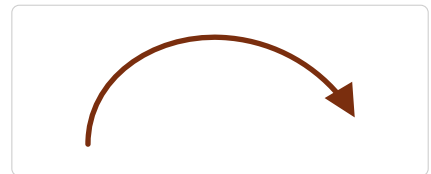
```
// one path with imposed directions, drawn with a broad nib
// un chemin à directions imposées, tracé à la plume plate
let p = mp-path("(0,0){dir 60}..(25,55){left}..(10,30){down}..(30,0){right}..(70,25){up}..(85,0)")
nib-stroke(p, pen: broadnib(9pt, t: 1.8pt, angle: 35deg), fill: rgb("#2b2b2b"), pad: 4pt)
```



A curved arrow

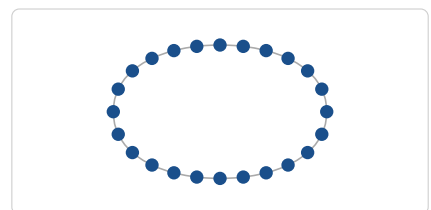
Shorten the stem so that it stops at the base of the head, then place the head with `frame-at` and `in-frame`:

```
let p = mp-path("(0,0){up}..(50,40)..(100,10)")
// arrowhead pointing along +x, tip at the origin
// pointe de flèche vers +x, sommet à l'origine
let head = polyline((-12pt, 6pt), (0pt, 0pt), (-12pt, -6pt)),
  cycle: true)
let end = frame-at(p, path-length(p))
mp-fig(
  draw(shorten(p, end: 10pt), pen: pencircle(2pt), fill: rgb("#7a2e0e")),
  mp-fill(in-frame(head, end), fill: rgb("#7a2e0e")),
  pad: 4pt)
```



A string of beads on a closed curve

```
let ring = mp-path("(0,0)..(40,25)..(80,0)..(40,-25)..cycle")
// 24 beads evenly spaced by arc length
// 24 perles régulièrement espacées en abscisse curviligne
let beads = frames-along(ring, count: 24).map(f =>
  mp-dot(f.pos, pen: pencircle(5pt), fill: rgb("#1a4f8b")))
mp-fig(
  draw(ring, pen: pencircle(0.6pt), fill: gray),
  ..beads, pad: 4pt)
```



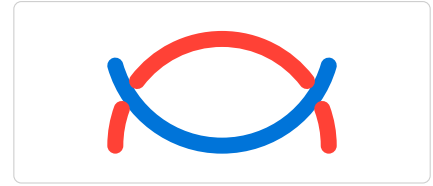
Over/under crossings by hand

`knot` does this automatically; here is the principle with `intersection-times` and `gap-at`:

```

let a = mp-path("(0,0)..(40,40)..(80,0)")
let b = mp-path("(0,30)..(40,0)..(80,30)")
let ts = intersection-times(a, b).map(x => x.at(0))
mp-fig(
  draw(b, pen: pencircle(6pt), fill: blue),
  ..gap-at(a, ts, 6pt).map(q =>
    draw(q, pen: pencircle(6pt), fill: red)),
  pad: 4pt)

```



Migrating a drawing written for nibst

Give the cubic segments to `cubics(..., y-down: true)` (`nibst` uses y-down coordinates) and translate the options: `width/thinness/angle/follow` → `nibpen`, `dash` → `dashes`, `pressure` → `pressure`, `layered` → `overlap: "layered"`, `debug` → `debug: true`.

Notes and limits

- **Hobby's algorithm** is validated against `mpost` on 17 test specifications (same control points within 1e-2 pt in the worst case, about 6e-5 otherwise); path operations match as well. The check script is in the repository (`tests/cmp_hobby.py`, requires MetaPost).
- Stroke shapes agree with brute-force stamping of the pen within 0.25 pt (tested at 576 ppi).
- Not implemented: `atleast`, `&` (path concatenation syntax — use `path-join`), `buildcycle`, and `...` (it behaves as `..`).
- `draw` returns **filled** shapes, not strokes; `mp-label` content is not part of the automatic bounding box.
- The plugin is a local WebAssembly file: it works with the Typst CLI, the web app and packages from Typst Universe, but not in environments that forbid WebAssembly plugins.
- **Performance.** A few hundred nodes are instantaneous; variable pens, dashes, pressure and `copperplate` cost more (subdivision by `refine`, one boolean union per piece). `knot` on a large area, or more than about a thousand curves in one document, can be slow and memory-hungry: lower `refine`, raise `tol`, enlarge `step`, or split the document.
- Errors start with `nib:` or `nibart:`; the most frequent: `pens` with the wrong number of entries, a length expected but a string or `em` given, a non-elliptical pen together with `dash/pressure`, `shorten` removing more than the whole path.

Credits and licence

MIT licence (see `LICENSE`). Third-party crates and acknowledgements: `THIRD-PARTY-NOTICES.md`. The path language and Hobby's algorithm come from Metafont/MetaPost (D. Knuth, J. D. Hobby); the calligraphic options are re-implemented from the behaviour described by the `nibst` package (B. Auguie) without reusing its code; the knot and ornament ideas follow the LaTeX packages `spath3`, `knots` and `calligraphy` (A. Stacey), also without sharing code.

Author: FERGOUS Abdelhak <@fergousA> — **repository:** <https://github.com/fergousA/nibart>.

Index of functions

arclength	12	weld	21
arctime	12	xscaled	24
brace-path	11	yscaled	24
broadnib	28		
close-smooth	10		
closest-time	15		
copperplate	40		
crossings	39		
cubic	6		
cubics	7		
dashes	30		
delimiter	41		
direction-angle	14		
direction-of	13		
draw	32		
fit-between	22		
frame-at	17		
frames-along	18		
gap-at	21		
in-frame	17		
intersection-point	16		
intersection-times	15		
join-smooth	10		
knot	38		
mp-dot	34		
mp-fig	36		
mp-fill	34		
mp-group	36		
mp-label	35		
mp-path	5		
mp-path-pts	5		
mp-skeleton	35		
nib-stroke	33		
nib-version	41		
nibpen	29		
offset	22		
paren-path	11		
path-bbox	14		
path-close	9		
path-join	8		
path-join-all	8		
path-length	12		
pencircle	27		
penellipse	27		
penpoly	28		
penrazor	29		
pensquare	28		
point-of	13		
polyline	7		
pressure	30		
prongs	40		
reflected	25		
remove-intervals	20		
reverse	19		
rotated	25		
round-corners	9		
scaled	23		
self-intersections	16		
shifted	23		
shorten	19		
slanted	24		
split-at	20		
straight	6		
stroke-items	32		
subpath	18		
transformed	26		